

Análisis del componente manejador de base de datos

Este diseño encapsula la persistencia de datos asegurando que la aplicación anfitriona interactúe con una interfaz genérica, manteniendo el desacoplamiento y cumpliendo con los objetivos de eficiencia de recursos propios del modelo GREENSOFT. A continuación, se detalla la documentación arquitectónica estructurada.

1. Contrato Base: i_database.hpp

```
#ifndef I_DATABASE_HPP
#define I_DATABASE_HPP

#include "i_component.hpp"
#include <string>
#include <vector>
#include <unordered_map>

// Representación genérica de una fila: clave (columna) -> valor (texto)
using DataRow = std::unordered_map<std::string, std::string>;
using ResultSet = std::vector<DataRow>;

class IDatabase : public IComponent
{
public:
    virtual ~IDatabase() noexcept = default;
    virtual ComponentResult connect(const std::string& connection_string) noexcept = 0;
    virtual void disconnect() noexcept = 0;
    virtual ComponentResult execute(const std::string& query, const
std::vector<std::string>& params) noexcept = 0;
    virtual ComponentResult query(const std::string& query, const
std::vector<std::string>& params, ResultSet& out_result) noexcept = 0;
};

#endif // I_DATABASE_HPP
```

Este archivo establece la abstracción pura para cualquier motor de base de datos que se desee implementar en el futuro:

- **Estructuras de Datos Ligeras (DataRow y ResultSet):** Se define DataRow como un alias de `std::unordered_map<std::string, std::string>` y ResultSet como un `std::vector<DataRow>`. Esta decisión evita la creación de clases polimórficas

pesadas para representar filas, garantizando un bajo consumo de memoria RAM al mapear la clave (nombre de la columna) directamente con su valor textual.

- **Firmas de Métodos y noexcept:** Las funciones `connect`, `disconnect`, `execute` y `query` están estrictamente marcadas con `noexcept`. Esto asegura que ninguna excepción generada por el motor de base de datos cruce la frontera del ABI.
- **Gestión de Memoria en Lecturas (query):** El método `query` no retorna el `ResultSet` por valor, sino que lo recibe como un parámetro de salida por referencia (`ResultSet& out_result`). Esto evita copias innecesarias en la pila y mantiene la compatibilidad de la firma con el contrato `noexcept`.

2. Implementación Concreta: `sqlite_component.cpp`

Esta clase materializa el contrato utilizando la C-API nativa de SQLite3, integrando mitigaciones críticas de seguridad y concurrencia.

Ciclo de Vida y Mitigación de Concurrencia

- `sqlite3_open(connection_string.c_str(), &db_)`: Inicializa la conexión instanciando el motor embebido directamente sobre el archivo local, sin requerir demonios de red.
- `sqlite3_busy_timeout(db_, 5000)`: Soluciona el problema intrínseco de concurrencia de SQLite. Instruye al motor para que, si un hilo encuentra la base de datos bloqueada por una escritura, el hilo espere pasivamente en el kernel hasta 5000 milisegundos en lugar de abortar la transacción inmediatamente con un error `SQLITE_BUSY`.
- `PRAGMA journal_mode=WAL;`: Es una optimización arquitectónica vital. Cambia el modo de registro tradicional a *Write-Ahead Logging* (WAL). Esto permite que las lecturas y escrituras ocurran de forma simultánea (lecturas no bloqueantes), escalando radicalmente el rendimiento en entornos multihilo.

Prevención de Inyección SQL y Gestión de Parámetros

- `prepare_and_bind` (Método Privado): Centraliza la compilación de sentencias SQL.
 - * `sqlite3_prepare_v2`: Compila la consulta SQL cruda (`query.c_str()`) en un objeto de código de bytes nativo de SQLite (`sqlite3_stmt`). Esto separa estructuralmente el código lógico de los datos ingresados por el usuario.

- **SQLITE_TRANSIENT**: Durante el bucle de enlace (`sqlite3_bind_text`), se pasa la bandera `SQLITE_TRANSIENT`. Esto le indica explícitamente a SQLite que no puede confiar en el ciclo de vida del puntero C++ original (`params[i].c_str()`), obligando al motor de la base de datos a realizar su propia copia privada del texto en memoria antes de que la variable C++ se destruya.

Ejecución y Extracción de Datos

- **Transacciones de Modificación (execute)**: Invoca `sqlite3_step(stmt)` y valida que el retorno sea estrictamente `SQLITE_DONE`, lo que confirma que la inserción, actualización o borrado finalizó sin interrupciones.
- **Transacciones de Lectura (query)**:
 - Invoca `out_result.clear()` para purgar cualquier dato residual en el vector de salida.
 - Utiliza un bucle `while (sqlite3_step(stmt) == SQLITE_ROW)` para iterar fila por fila de forma eficiente.
 - Por cada columna detectada mediante `sqlite3_column_count`, extrae el nombre (`sqlite3_column_name`) y el valor (`sqlite3_column_text`), insertándolos en el mapa `DataRow`.
 - Se previene la desreferenciación de punteros nulos en campos vacíos de la base de datos utilizando el operador ternario `col_value ? col_value : "null"`.
- **sqlite3_finalize(stmt)**: En todos los casos, el objeto compilado se destruye para evitar fugas de memoria nativa.

3. Exportación de la Interfaz Binaria (C-API)

El bloque final del archivo asegura que el componente pueda ser cargado dinámicamente por la infraestructura del Host.

- **extern "C"**: Desactiva el *name mangling* del compilador de C++, garantizando que los símbolos binarios (como `create_component`) mantengan un nombre predecible y estandarizado para ser enlazados mediante `dlsym()`.
- El bloque de instanciación captura cualquier error grave de reserva de memoria (`try { return new SqliteComponent(); } catch (...)`) para evitar caídas catastróficas, devolviendo un puntero seguro que el `ModuleManager` evaluará.